Visione Artificiale

II parte:
MACHINE LEARNING vs COMPUTER VISION

Raffaella Lanzarotti

Where are we?

First part: Image formation and Early vision

- Image formation
  - Geometric Camera Models
  - Color spaces
- Image Processing
  - Punctual and spatial processing
  - Feature Extraction
- Reconstruction
  - Camera calibration
  - Stereo Vision
  - Structure from Motion and RGB-d Cameras
  - Optical flow and Tracking

Second part: Machine learning for CV

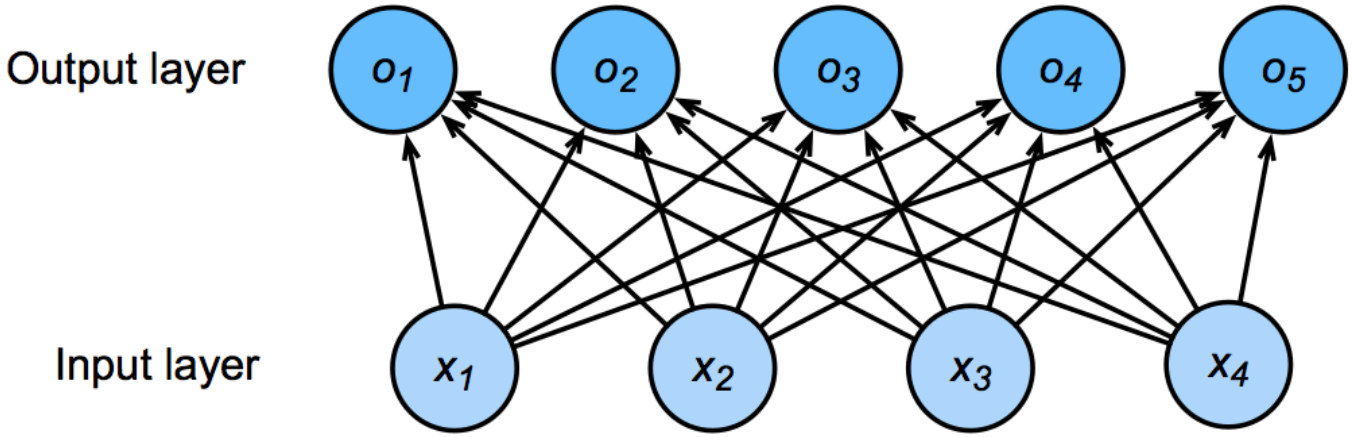
- Linear Neural Network
- **Multi Layer Perceptron**
- Convolutional Neural Networks
- Recurrent Neural Networks
- Transformers
- Graph Neural Networks
- Generative Adversarial Networks

MULTILAYER PERCEPTRONS

Chapters 6-9 Deep Learning (Bishop)

Chapter 4, Dive into deep learning

Limits of one layer NN

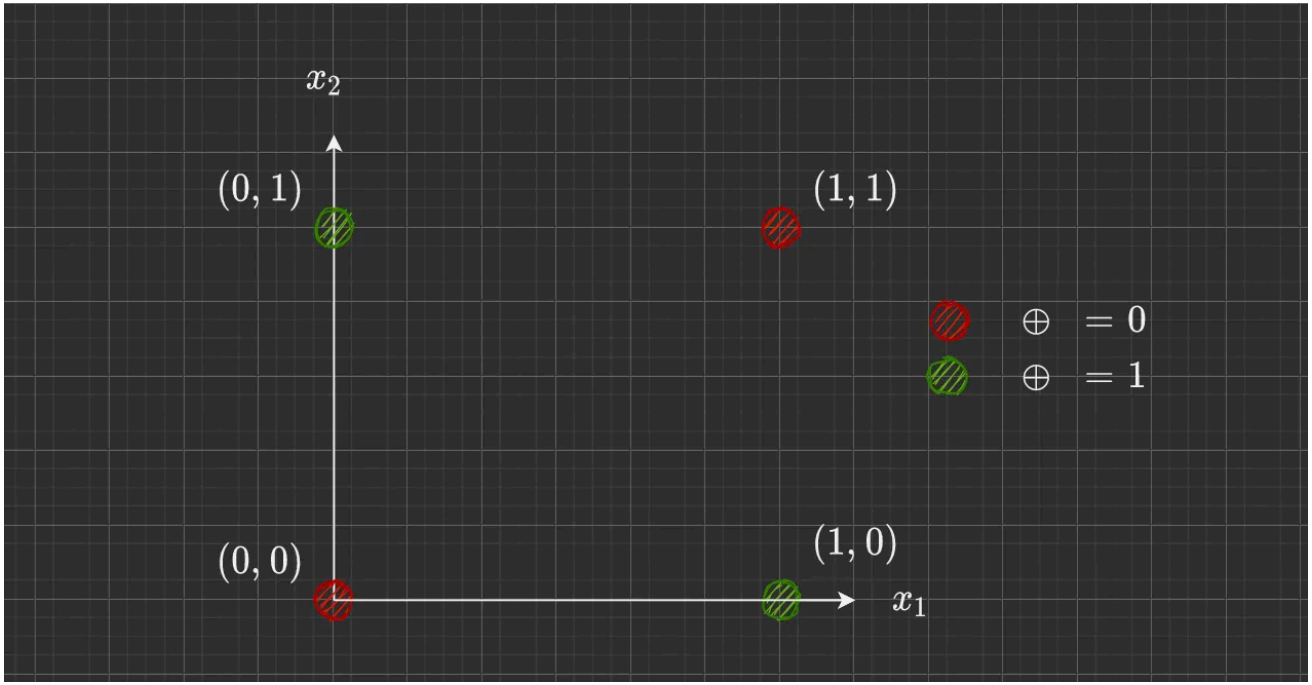


Softmax regression:
map input directly to the output via a single affine transformation + softmax operation.

Limit: assumption of linearity in affine transformation
Work only for linearly separable classes
Need for more expressive models

Learning XOR

Data	Target
[0, 0]	0
[0, 1]	1
[1, 0]	1
[1, 1]	0

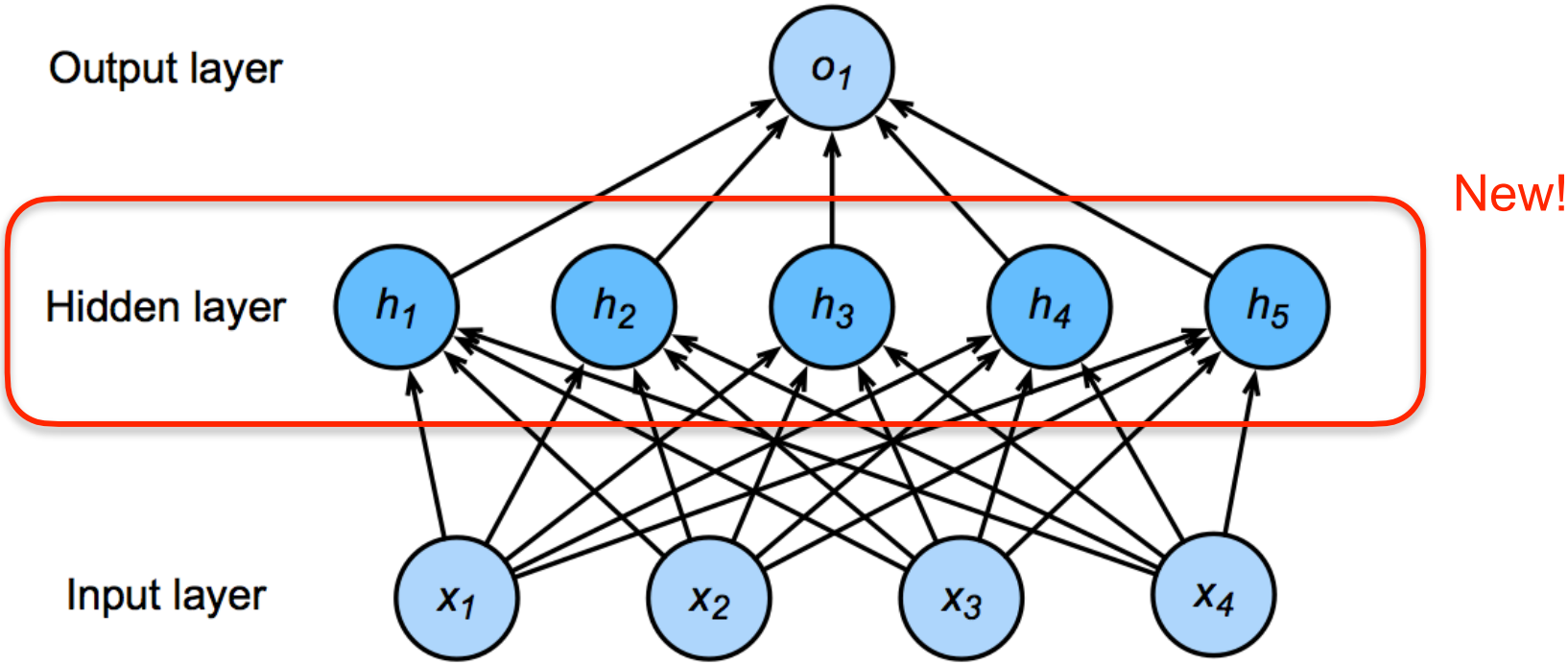


XOR: No solution with linear model: they assume monotonicity!

https://towardsdatascience.com/how-neural-networks-solve-the-xor-problem-59763136bdd7

Single Hidden Layer for Linear regression

Does introducing hidden layer(s) help?



Single Hidden Layer

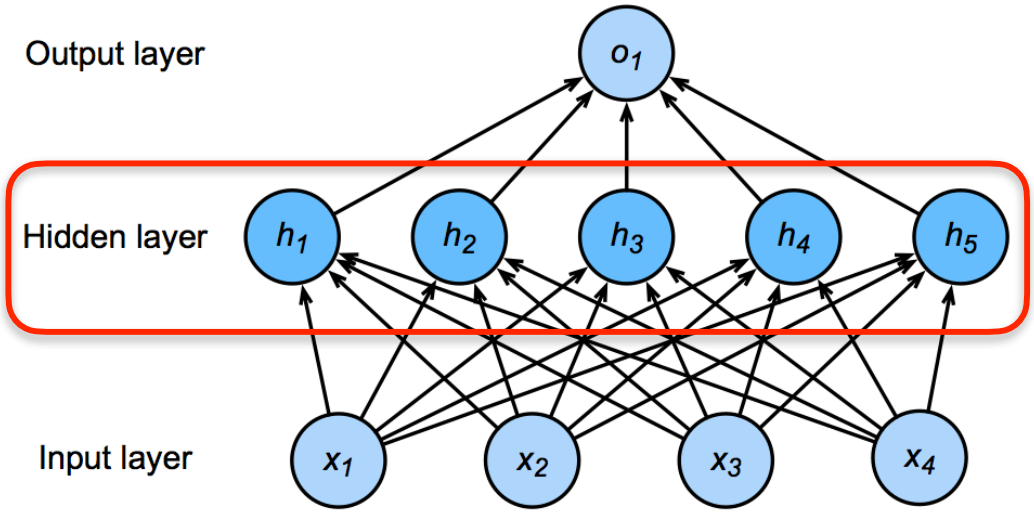
Q: Does introducing hidden layer(s) help?

- Input $\mathbf{x} \in \mathbb{R}^n$
- Hidden $\mathbf{W}_1 \in \mathbb{R}^{m \times n}, \mathbf{b}_1 \in \mathbb{R}^m$
- Output $\mathbf{w}_2 \in \mathbb{R}^m, b_2 \in \mathbb{R}$

$$\mathbf{h} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1$$

$$\mathbf{o} = \mathbf{w}_2^T \mathbf{h} + b_2$$

hence $o = \mathbf{w}_2^\top \mathbf{W}_1 \mathbf{x} + b'$



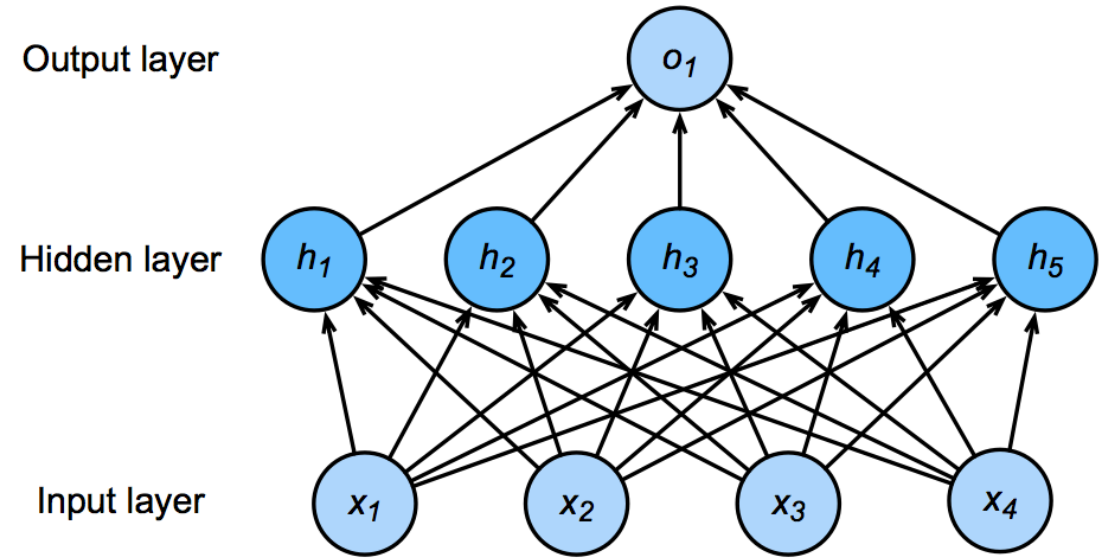
A: NO, combining several linear layers we still get a linear model (with some unobserved variables)

Single Hidden Layer

We need a nonlinear activation function

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{w}_2^T\mathbf{h} + b_2$$



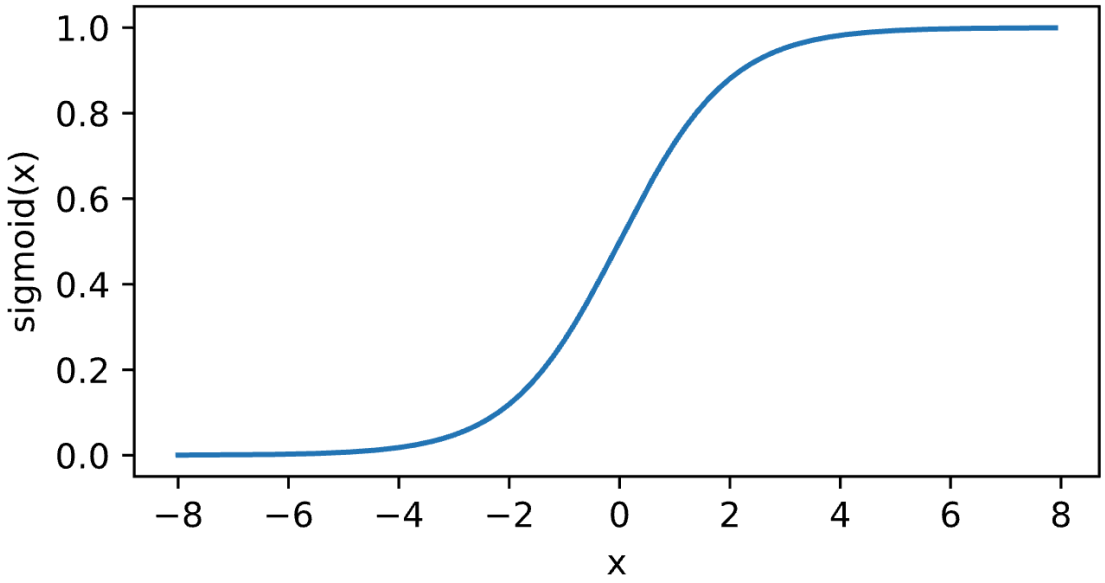
New! σ is an element-wise activation function

Multilayer Perceptron:
loosely refers to NN with *fully connected layers* and 1 or more hidden layers, especially with non linear activation function

Sigmoid Activation

Map input into (0, 1), a soft version of $\sigma(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$

$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}$$



ADVANTAGES:

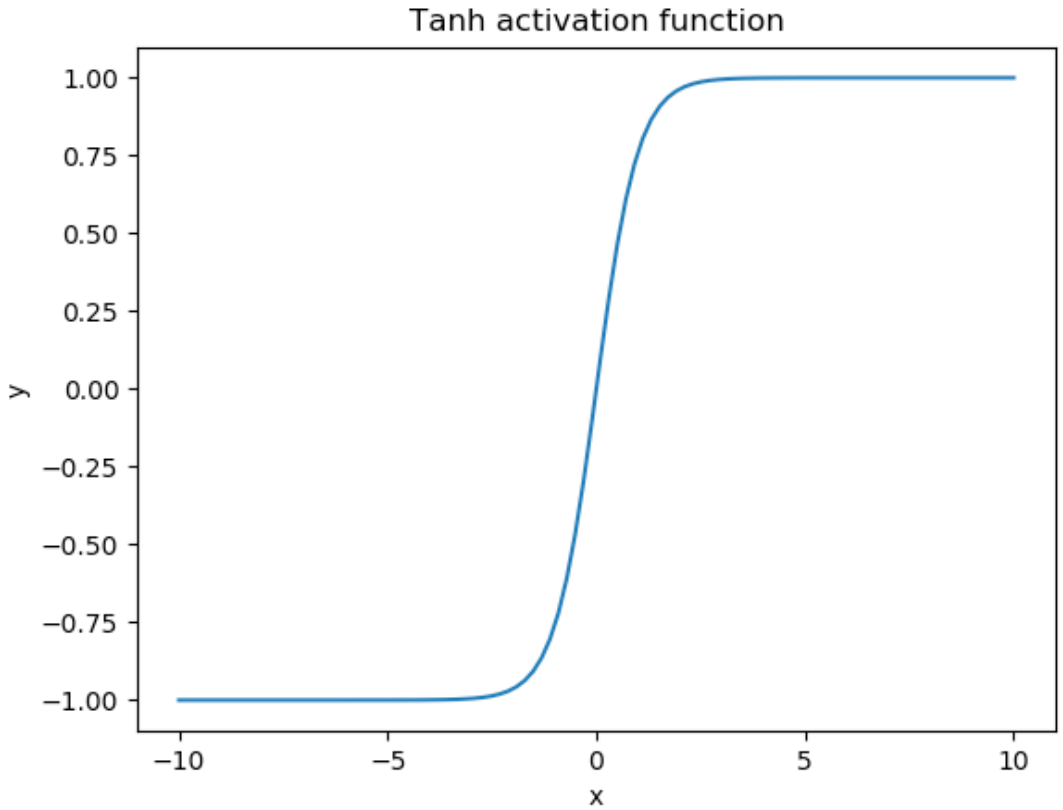
- Limited range (0;1) (ideal for binary classification)
- Simple gradient

DISADVANTAGES:

- Vanishing gradient (not suitable for deep network)
- Not symmetric wrt 0

Tanh Activation

Map input into (-1, 1): $\tanh(x) = \dfrac{(e^{x} - e^{-x})}{(e^{x} + e^{-x})}$



ADVANTAGES:

- Limited range (-1;1)
- symmetric wrt zero

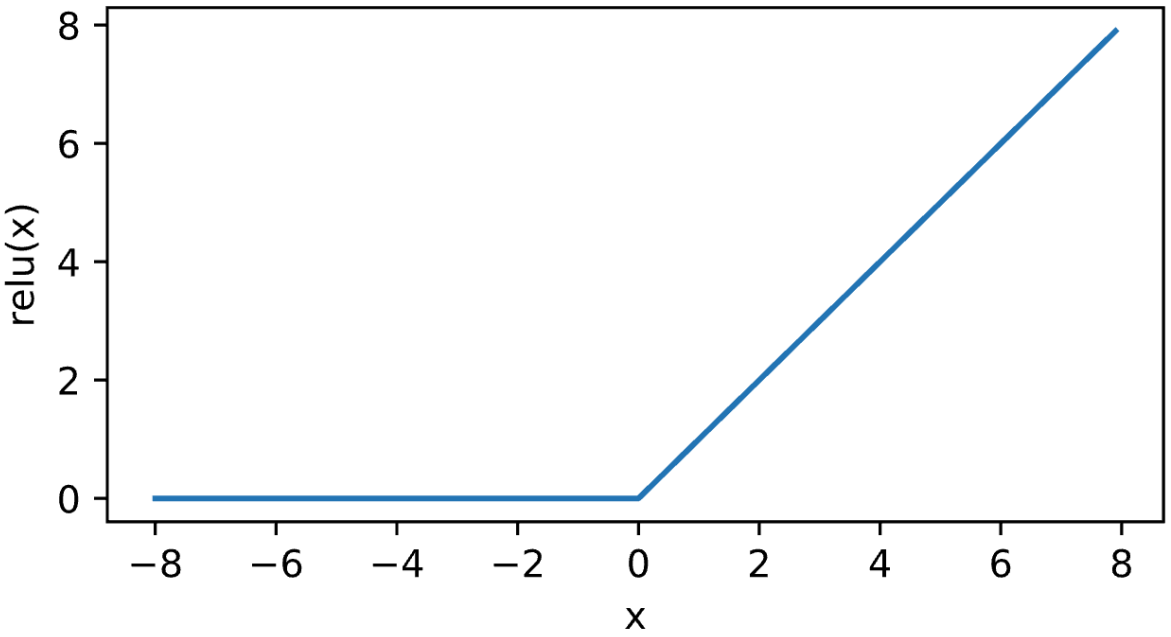
DISADVANTAGES:

- Vanishing gradient
  (not suitable for deep network)

ReLU Activation

ReLU: rectified linear unit

$$\mathrm{ReLU}(x) = \max(x, 0)$$



ADVANTAGES:

- Simple
- Avoid vanishing gradient
- Produces a sparse output

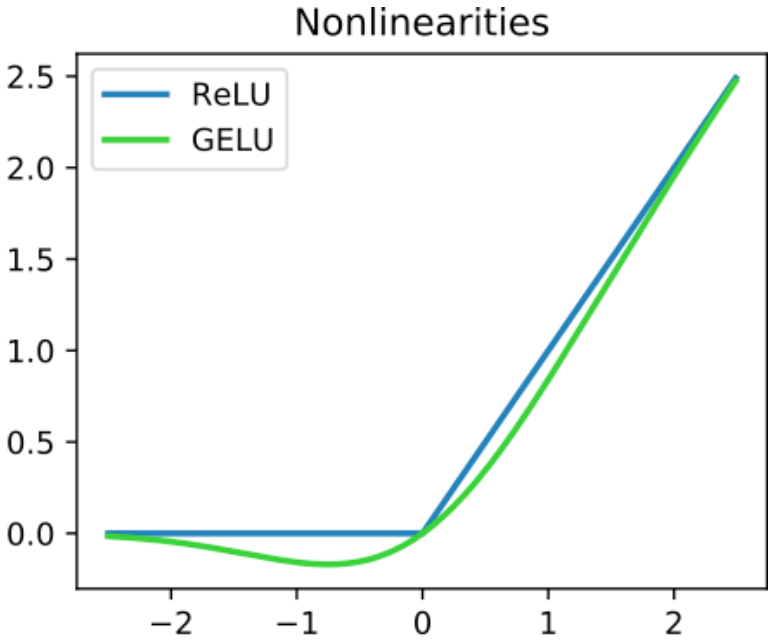
DISADVANTAGES:

- Put to zero several neurons
- Not symmetric wrt 0
- Not differentiable in zero (adopt the weak derivative or differentiable variants of ReLU, eg: Leaky ReLU, GeLU)

GeLU Activation

GeLU: Gaussian Error Linear Units (Hendrycks and Gimpel 2018)

$$GELU(x) = \frac{x}{2} \cdot (1 + \tanh(\sqrt{\frac{2}{\pi}} \cdot (x + 0.044715 \cdot x^3)))$$



ADVANTAGES:

- Continue and differentiable
- Smoother than ReLU

DISADVANTAGES:

- complex computation

https://en.wikipedia.org/wiki/Rectifier_%28neural_networks%29

Multi-class Classification, with single hidden layer

- Input

$$\mathbf{x} \in \mathbb{R}^n$$

- Hidden

$$\mathbf{W}_1 \in \mathbb{R}^{m \times n} \text{ and } \mathbf{b}_1 \in \mathbb{R}^m$$

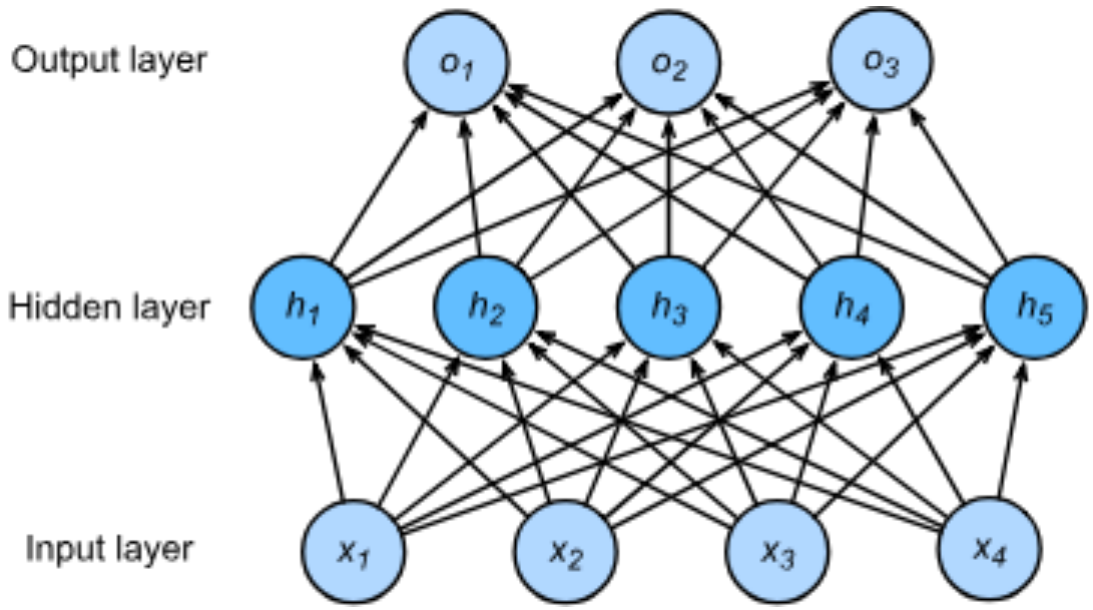
$$\mathbf{W}_2 \in \mathbb{R}^{m \times d} \text{ and } \mathbf{b}_2 \in \mathbb{R}^d$$

- Output

$$\mathbf{h} = \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{w}_2^T \mathbf{h} + \mathbf{b}_2$$

$$\mathbf{y} = \text{softmax}(\mathbf{o})$$



Single hidden layer vs Deep networks

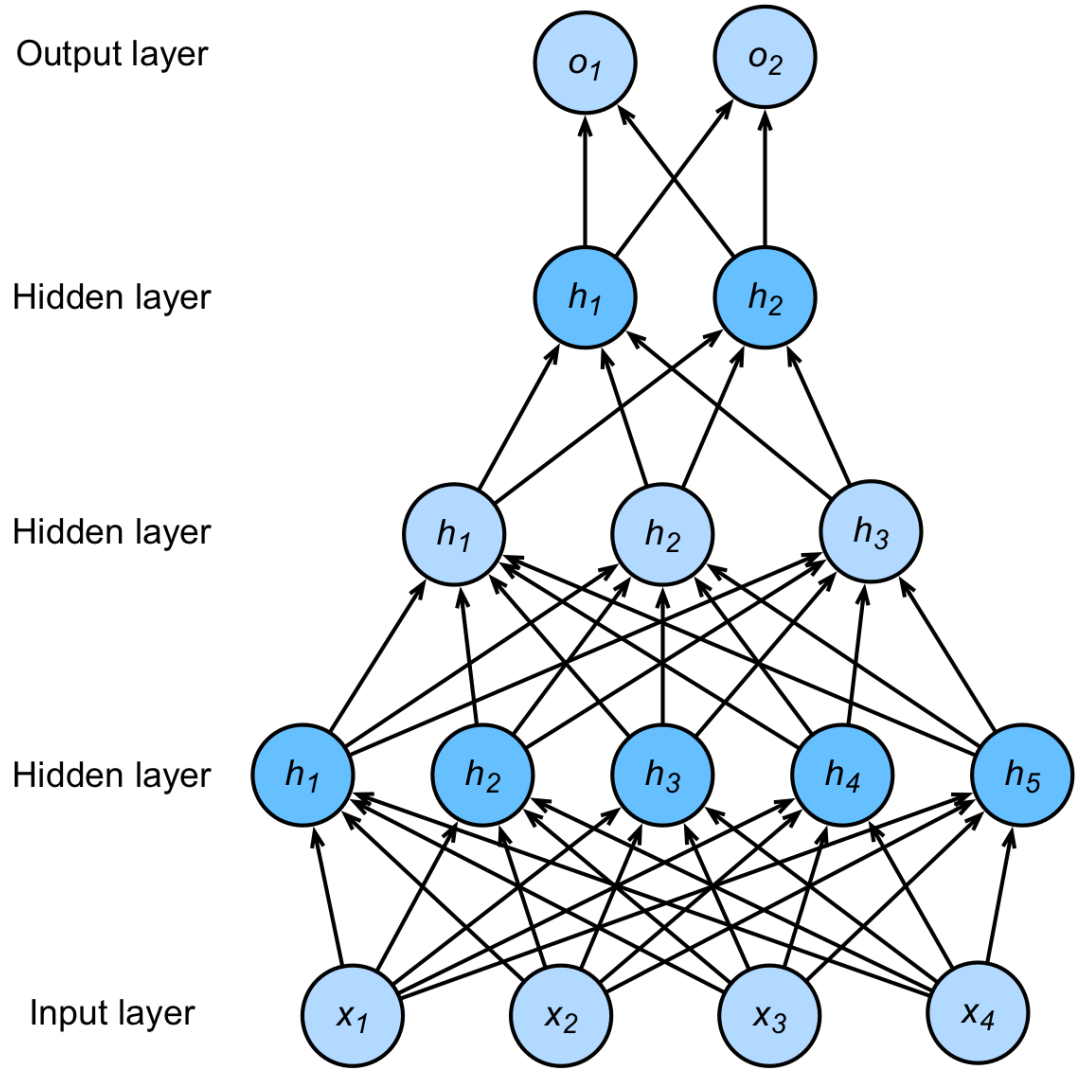
- **Universal Approximation Theorem (G. Cybenko, 1989):**

  Single hidden layer networks (with sufficient nodes) with non linear activation functions can model any continuous function with arbitrary precision

- **WHY DEEP NETWORKS?**
  - the theorem prove the existence but not the way to get it
  - they can learn **features hierarchically**, extracting increasingly complex features
  - they are **more efficient** allowing parallel processing for training

Multiple Hidden Layers

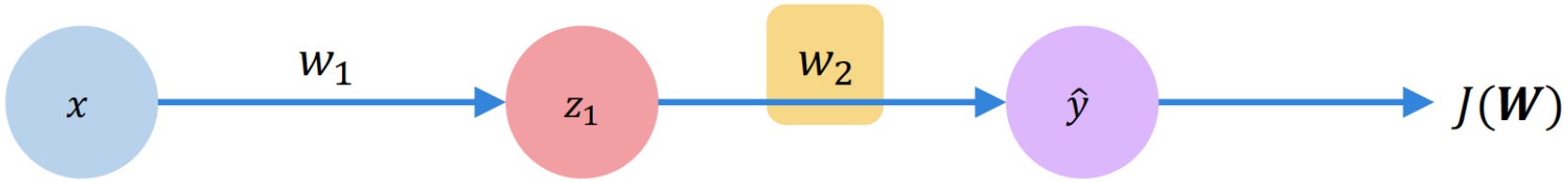
$$\mathbf{h}_1 = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$\mathbf{h}_2 = \sigma(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2)$$
$$\mathbf{h}_3 = \sigma(\mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3)$$
$$\mathbf{o} = \mathbf{W}_4\mathbf{h}_3 + \mathbf{b}_4$$



Hyper-parameters:

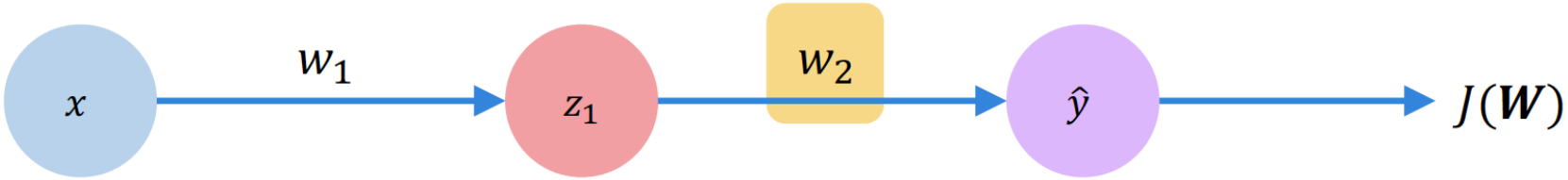
- # of hidden layers
- Size for each hidden layer

Computing Gradients: Backpropagation



How does a small change in one weight (ex. $\boldsymbol{w_2}$) affect the final loss $\boldsymbol{J(W)}$?

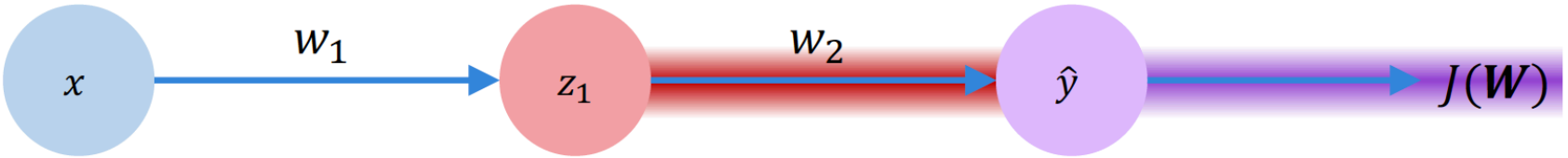
Computing Gradients: Backpropagation



$$\frac{\partial J(\boldsymbol{W})}{\partial w_2} =$$

Let's use the chain rule!

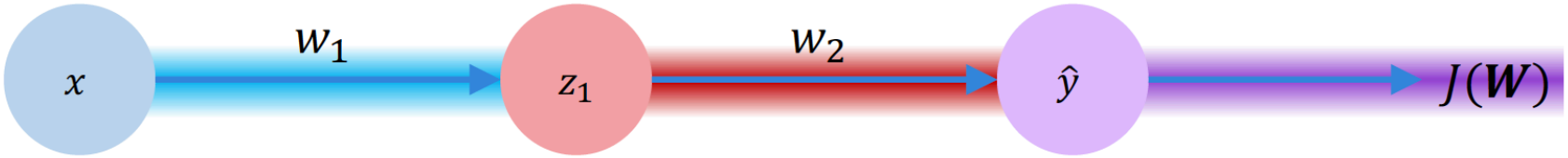
Computing Gradients: Backpropagation



$$\frac{\partial J(\boldsymbol{W})}{\partial w_2} = \frac{\partial J(\boldsymbol{W})}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_2}$$

Computing Gradients: Backpropagation

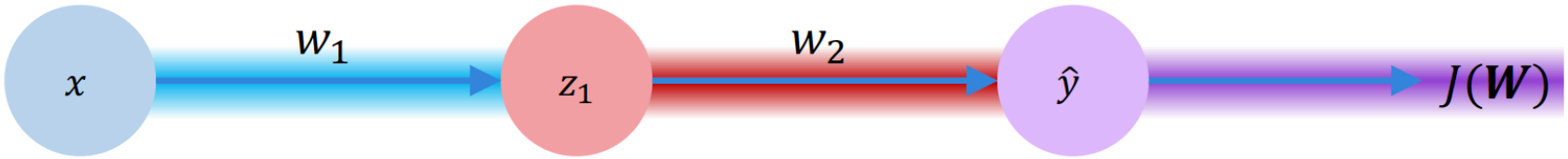
Going deeper…



$$\frac{\partial J(\boldsymbol{W})}{\partial w_1} = \frac{\partial J(\boldsymbol{W})}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_1} * \frac{\partial z_1}{\partial w_1}$$

Computing Gradients: Backpropagation

Going deeper…



$$\frac{\partial J(\boldsymbol{W})}{\partial w_1} = \frac{\partial J(\boldsymbol{W})}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_1} * \frac{\partial z_1}{\partial w_1}$$

Repeat this for ***every weight in the network*** *using gradients from later layers*

Gradient descent optimization algorithms

- Gradient descent:
  - batch [gradient on the whole Training Set for one param update]
  - stochastic [gradient on the single Training sample for one param update]
  - *mini-batch** [update for every mini-batch of n training examples]

- Challenges:
  - Choosing a proper learning rate
  - Adopting learning rate schedules
  - Adopting different learning rate according to the feature frequency (eg: larger update for rarely occurring features)
  - Avoiding getting trapped in suboptimal local minima

https://ruder.io/optimizing-gradient-descent/

Gradient descent optimization algorithms

- **Momentum:**

  helps accelerate SGD in the relevant direction and dampens oscillations

Image 2: SGD without momentum

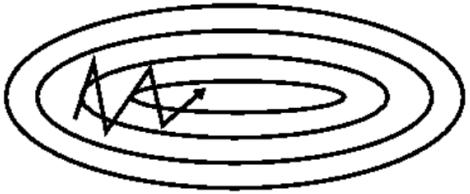


Image 3: SGD with momentum

Momentum can be seen as a ball running down a slope

$$v_t = \eta \frac{\partial L_t(\theta)}{\partial \theta} + \gamma v_{t-1}$$

$$\theta_t = \theta_{t-1} - v_t$$

v_t : velocity

NB: for $\gamma = 0$ it is the standard SGD

Analytically…

Gradient descent optimization algorithms

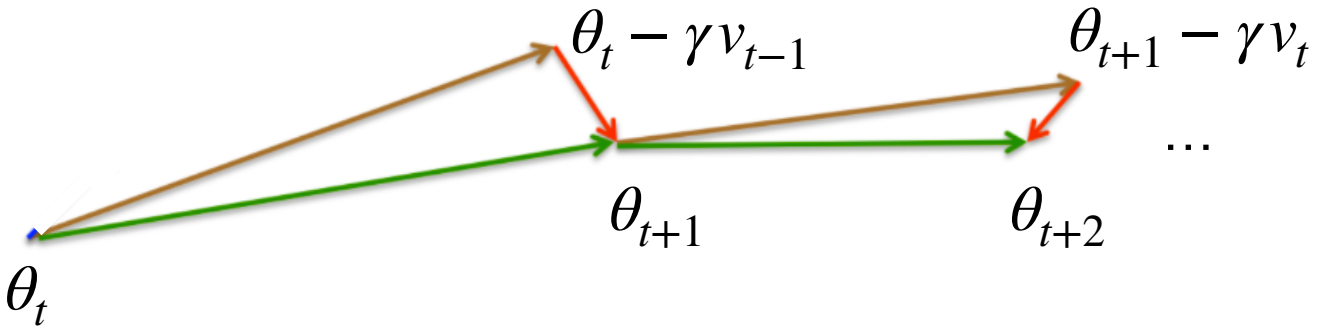
- **Nesterov accelerated gradient:**

Calculating the gradient not w.r.t. to our current parameters θ but w.r.t. the approximate future position of our parameters

$$v_t = \gamma v_{t-1} + \eta \frac{\partial L(\theta - \gamma v_{t-1})}{\partial \theta}$$

$$\theta_t = \theta_{t-1} - v_t$$

Graphically:



Gradient descent optimization algorithms

- Adagrad: different learning rate for each parameter in the network

  - Init: θ (parameter vector), r (accumulator of squared gradients)
  - Set hyperparameter: η (learning rate)

  - At each iteration t, for each parameter:
    - Compute the gradient of the loss function wrt the parameter vector: $g_t = \frac{\partial L(\theta_t)}{\partial \theta}$
    - Accumulate the sum of squared gradients for each parameter: $r_t = r_{t-1} + g_t^2$
    - Update each parameter as: $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{r_t} + \epsilon} \cdot g_t$

Gradient descent optimization algorithms

- Adagrad:
  - **different learning rate for each parameter in the network**
  - learning rates **inversely proportional to the sum of past gradients**:
    - **small update** for frequently occurring features (or with large gradient)
    - **large update** for less frequent features (or with small gradient)

  - Advantages:
    - the optimizer converges faster and more reliably than using a fixed learning rate for all parameters.
    - well-suited for dealing with sparse data or when there is a wide variation in the magnitude of the gradients across different parameters

  - Disadvantage:
    - learning rates can become very small as the sum of squared gradients grows larger, which can cause the algorithm to stop making significant updates

Gradient descent optimization algorithms

- RMSProp:
  - As Adagrad, different learning rate for each parameter in the network
  - learning rates divided by a moving average of the magnitudes of recent gradients for each weight

    - Init: θ (parameter vector), r (moving average of the squared gradients)
    - Set hyperparameter: η (learning rate) and β (decay rate)
    - At each iteration t, for each parameter:,
      - compute the gradient $g_t = \frac{\partial L(\theta_t)}{\partial \theta}$
      - Update r s.t.: $r_t = \beta \cdot r_{t-1} + (1-\beta) \cdot g_t^2$
      - Update the parameter: $\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{r_t} + \epsilon} g_t$ (ϵ to avoid division by zero)

Gradient descent optimization algorithms

- Adam (Adaptive Moment Estimation):
  - combines Adagrad and RMSProp
  - adaptive learning rates for each parameter
  - exponentially decaying average of past gradient and squared gradients v_t

Adam behaves like a heavy ball with friction, which thus prefers flat minima in the error surface

Analytically…

Gradient descent optimization algorithms

- Adam:

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t.

Require: α: Stepsize
Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates
Require: $f(\theta)$: Stochastic objective function with parameters θ
Require: θ_0: Initial parameter vector
$m_0 \leftarrow 0$ (Initialize 1st moment vector)
$v_0 \leftarrow 0$ (Initialize 2nd moment vector)
$t \leftarrow 0$ (Initialize timestep)
while θ_t not converged **do**
$t \leftarrow t + 1$
$g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)
$\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)
$\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)
$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$ (Update parameters)
end while
return θ_t (Resulting parameters)

Kingma, D. P., & Ba, J. L. (2015). Adam: a Method for Stochastic Optimization. International Conference on Learning Representations, 1–13

Training Error and Generalization Error

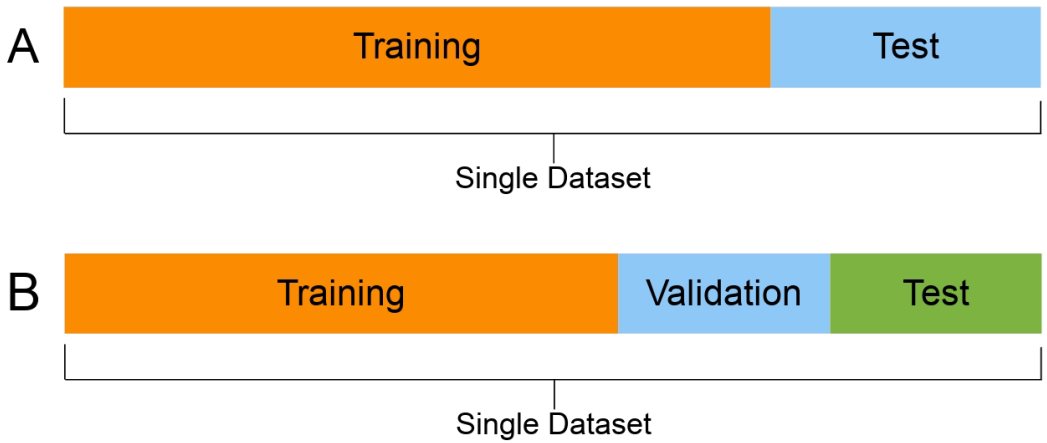
- Training error: model error on the training data
- Generalization error: model error on **new** data
  - Overfitting: fitting the training data more than the underlying distribution

Bad generalization capability!

Validation and Test Dataset

- Validation dataset: a dataset (not mixed with the training data) used to:
  - select the hyperparameters*:* how many hidden layers? which activation function? …
  - evaluate the model's generalization ability

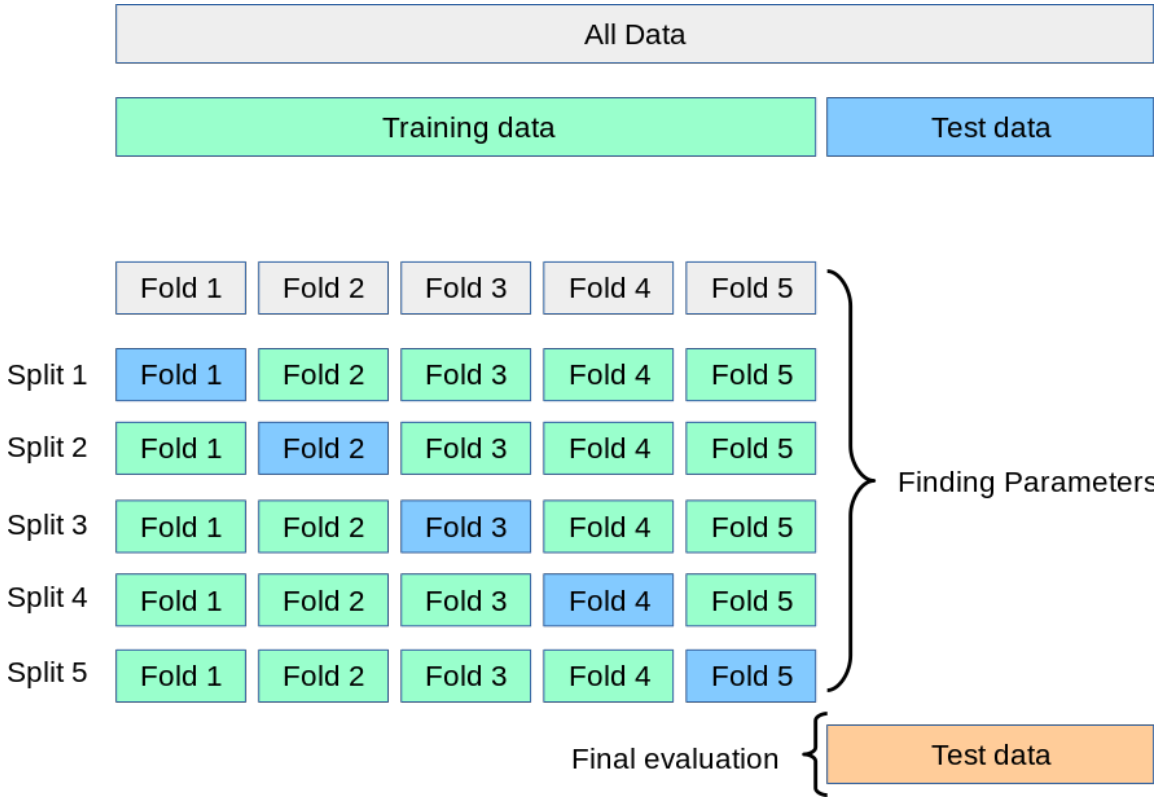
- Test dataset: for the final evaluation of the model performance
  - not used during model conception and training
  - separate from training and validation set



K-fold Cross Validation

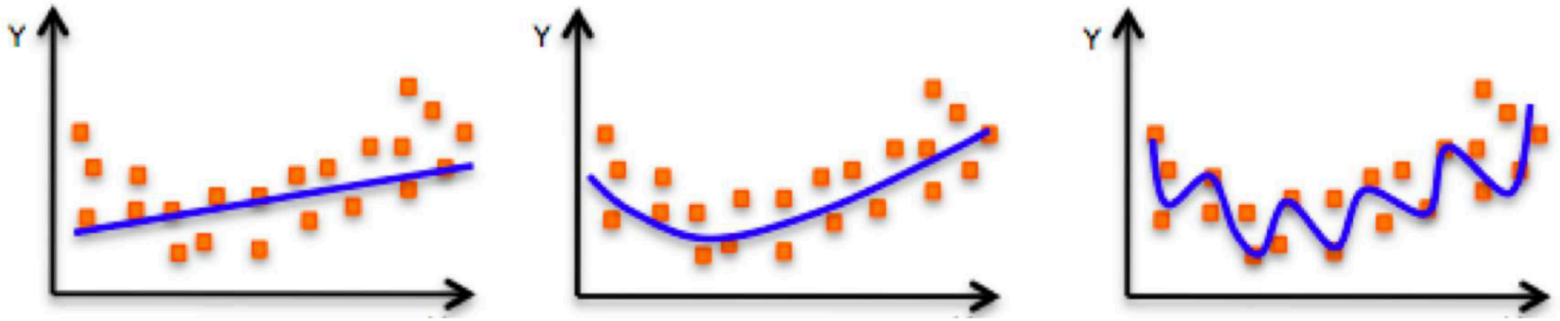
- When not sufficient data

- Method:
  - Partition the training data into *K* parts
  - For *i = 1, …, K*
    - i^{th} part = validation set,
    - the rest for training
  - Report the average over the *K* validation errors

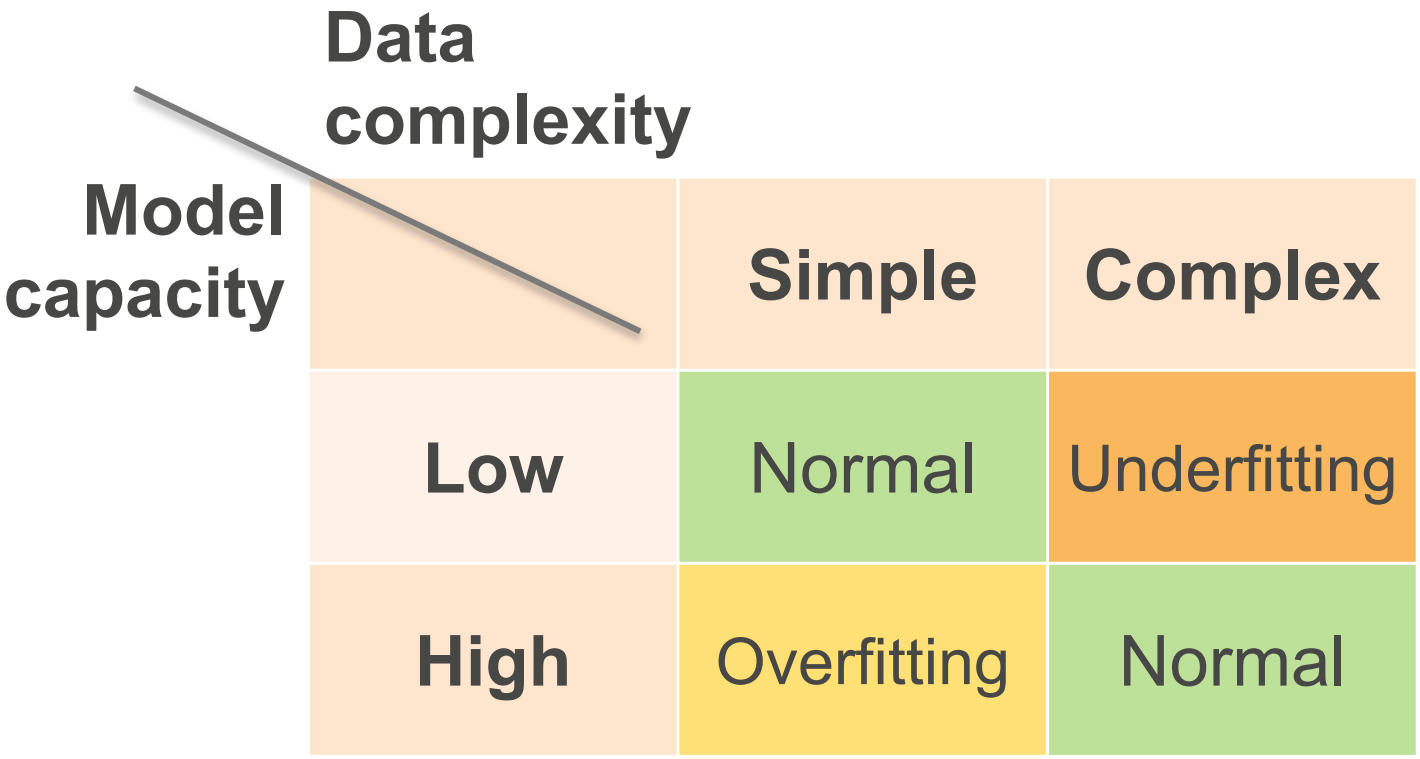


- Popular choices:
  K = 5 or 10

Underfitting
Overfitting



Underfitting and Overfitting

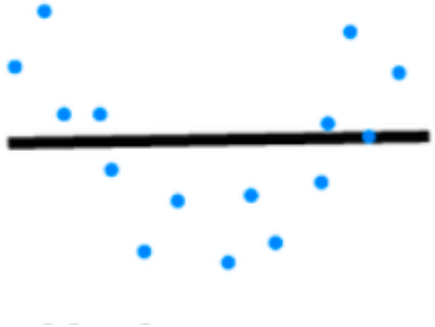


Model Capacity

The ability to fit variety of functions

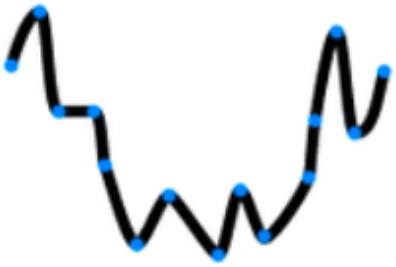
- **Underfitting**

  Low capacity models struggles to fit training set (too simple model)

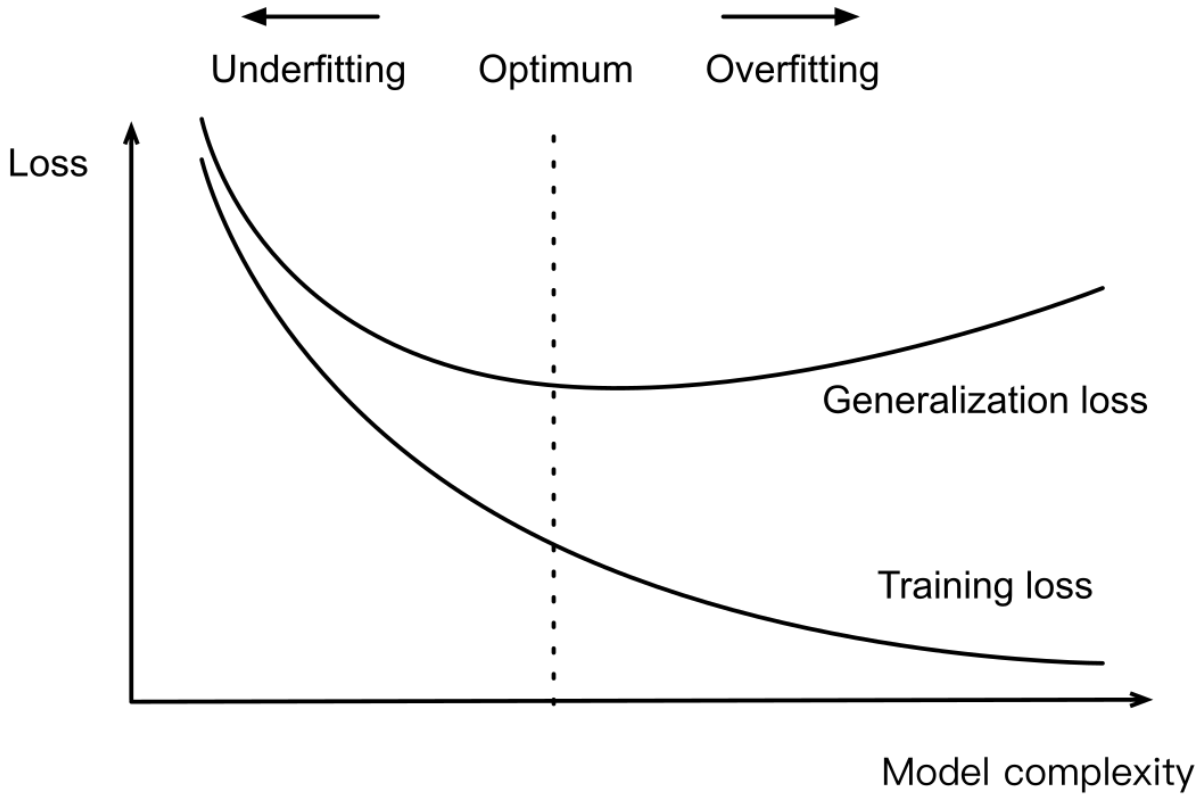


- **Overfitting**

  High capacity models can memorize the training set (training error significantly lower than validation error)



Influence of Model Complexity



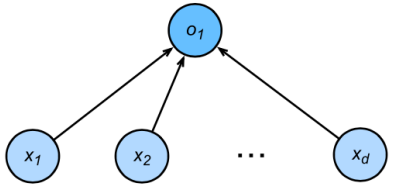
Estimate Model Capacity

- It's hard to compare complexity between different algorithms
  - e.g. tree vs neural network

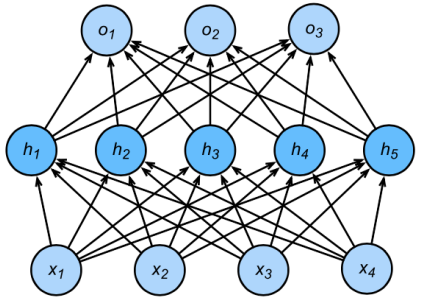
- Given an algorithm family, two main factors matter:
  - The *number* of parameters (nb of layers and of neurons)
  - The *values* taken by each parameter (eg non-zero parameters)

- It holds the Occam's razor principle: simpler models are preferred over more complex ones, given the same performances

$d + 1$

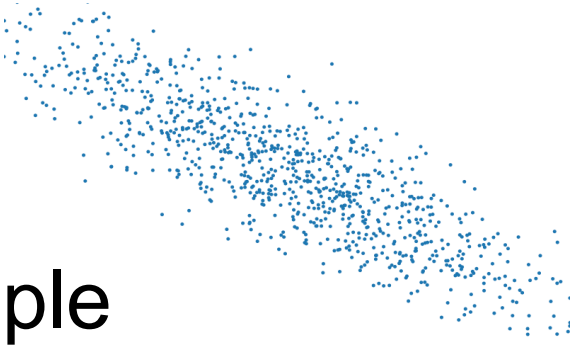


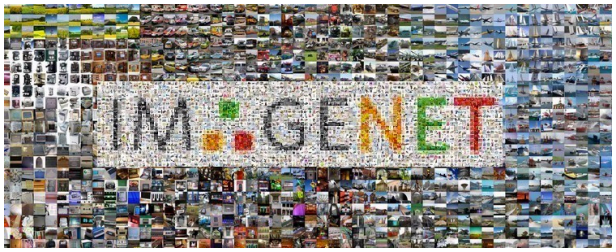
$(d + 1)m + (m + 1)k$



Data Complexity

- Multiple factors matters
  - # of examples
  - # of elements in each example
  - time/space structure
  - diversity





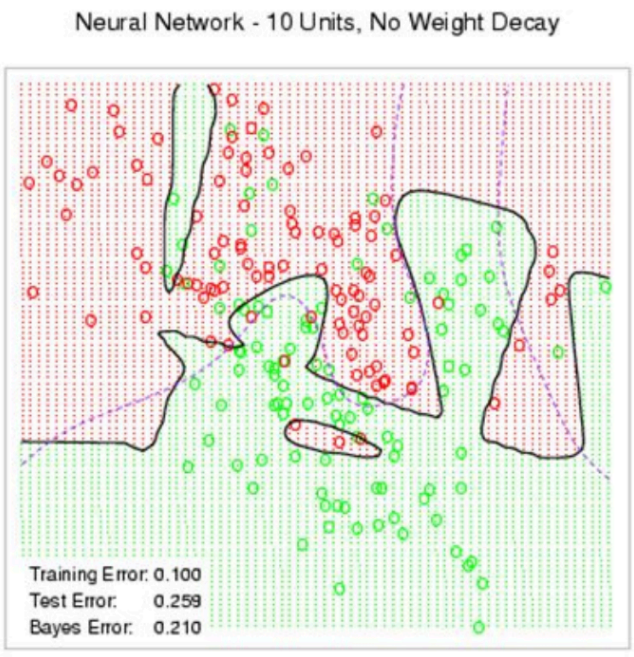
Regularization

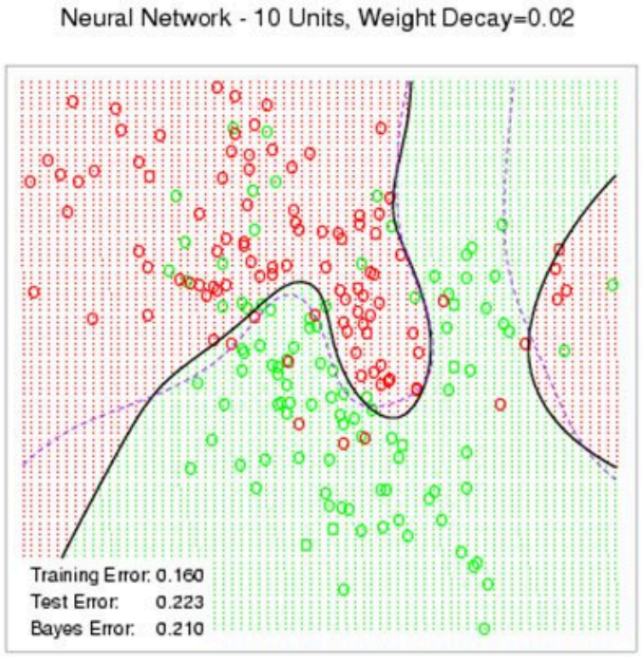
- What is it?

  Technique that constrains our optimization problem to discourage complex models

- Why do we need it?
  - avoid overfitting
  - Improve generalization of our model on unseen data

Weight Decay



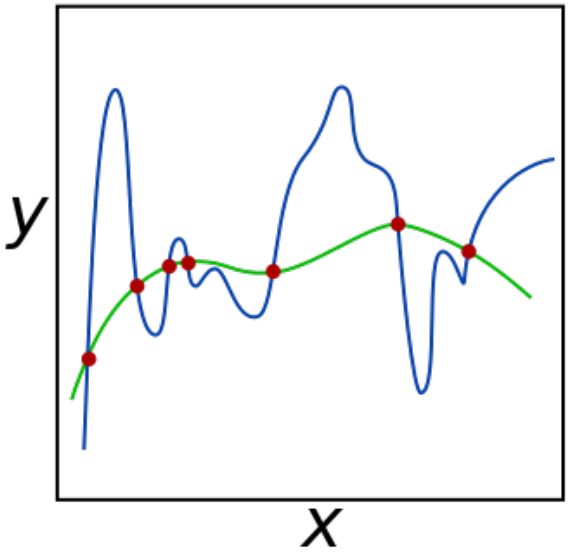


Squared Norm Regularization as Hard Constraint

- Weight decay or L_2 regularization: technique for regularizing parametric ML model:
  - A small $\mathbf{w}$ means more regularization
  - Reduce model complexity by limiting value range

    $\min \; L(\mathbf{w}, b) \quad$ subject to $\quad \|\mathbf{w}\|^2 \leq \theta$

  - Often do not regularize bias b (would produce little difference in practice)



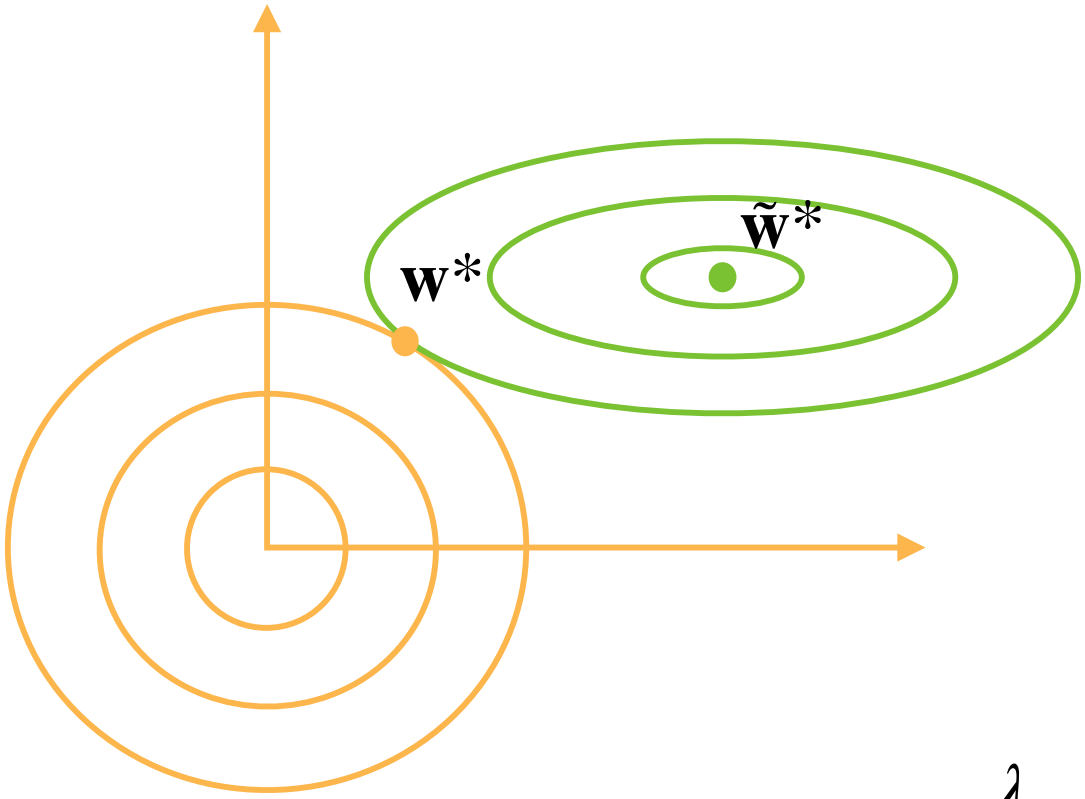
Squared Norm Regularization as Soft Constraint

- For each $\mathbf{w}$, we can find λ to rewrite the hard constraint version into a formulation with **penalization**:

$$\min L(\mathbf{w}, b) + \frac{\lambda}{2}\|\mathbf{w}\|^2$$

  - Hyper-parameter λ controls regularization importance
    - $\lambda = 0$: no regularization
    - $\lambda \to \infty, \mathbf{w}^* \to \mathbf{0}$ (shrinkage of optimal solution to 0)

Illustrate the Effect on Optimal Solutions



$$\tilde{\mathbf{w}}^* = \arg\min \; L(\mathbf{w}, b)$$

$$\mathbf{w}^* = \arg\min \; L(\mathbf{w}, b) + \frac{\lambda}{2}\|\mathbf{w}\|^2$$

Update Rule

- Compute the gradient

$$\frac{\partial}{\partial \mathbf{w}} \left(L(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|^2 \right) = \frac{\partial L(\mathbf{w}, b)}{\partial \mathbf{w}} + \lambda \mathbf{w}$$

- Update weight at time t

$$\mathbf{w}_{t+1} = (1 - \eta\lambda)\mathbf{w}_t - \eta \frac{\partial L(\mathbf{w}_t, b_t)}{\partial \mathbf{w}_t}$$

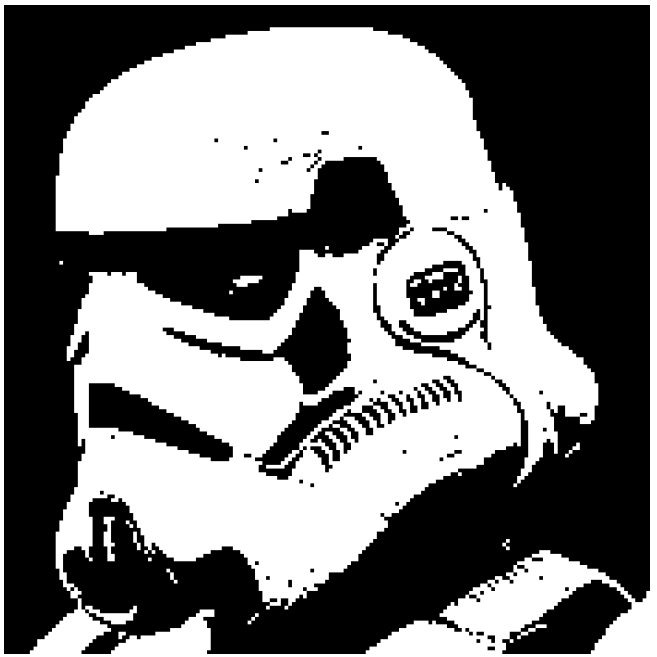
  - Often $\eta\lambda < 1$, so also called weight decay in deep learning

Notebook: `03_weight_decay.ipynb`

Dropout

Motivation

- A good model should be robust under modest changes in the input
  - Dropout: inject noises into internal layers, while preserving expectation of intermediate activations



Dropout: Add Noise without Bias

- **Dropout**: at each iteration perturbs each i-th element tossing a Bernulli$(1-p)$ coin (p : dropout probability)

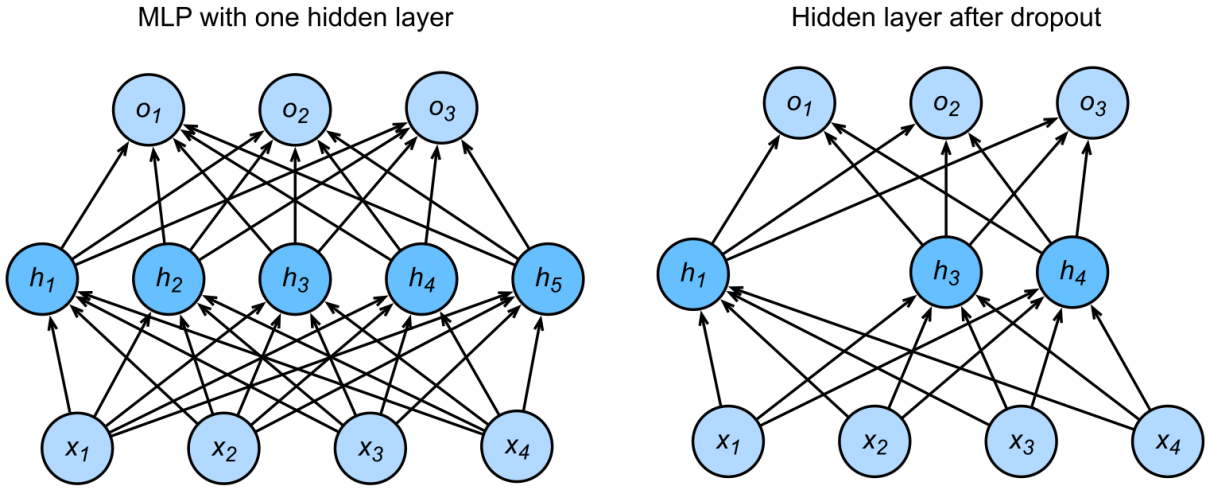
$$x_i' = \begin{cases} 0 & \text{with probablity } p \\ \frac{x_i}{1-p} & \text{otherise} \end{cases}$$

  - equivalent to retain a **subset** of layer's neurons
  - the remaining neurons are forced to learn more robust features
  - No dropout in inference (testing, prediction)

Dropout

- Often apply dropout on the output of hidden fully-connected layers

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$\mathbf{h}' = \text{dropout}(\mathbf{h})$$
$$\mathbf{o} = \mathbf{W}_2\mathbf{h}' + \mathbf{b}_2$$
$$\mathbf{y} = \text{softmax}(o)$$



Notebook: `04_dropout.ipynb`

Early stopping

Regularization: early stopping

- Stop training before we have a chance to overfit

Overfitting:
when a model starts to perform worse on the test (validation) set than on the training set

Regularization: early stopping

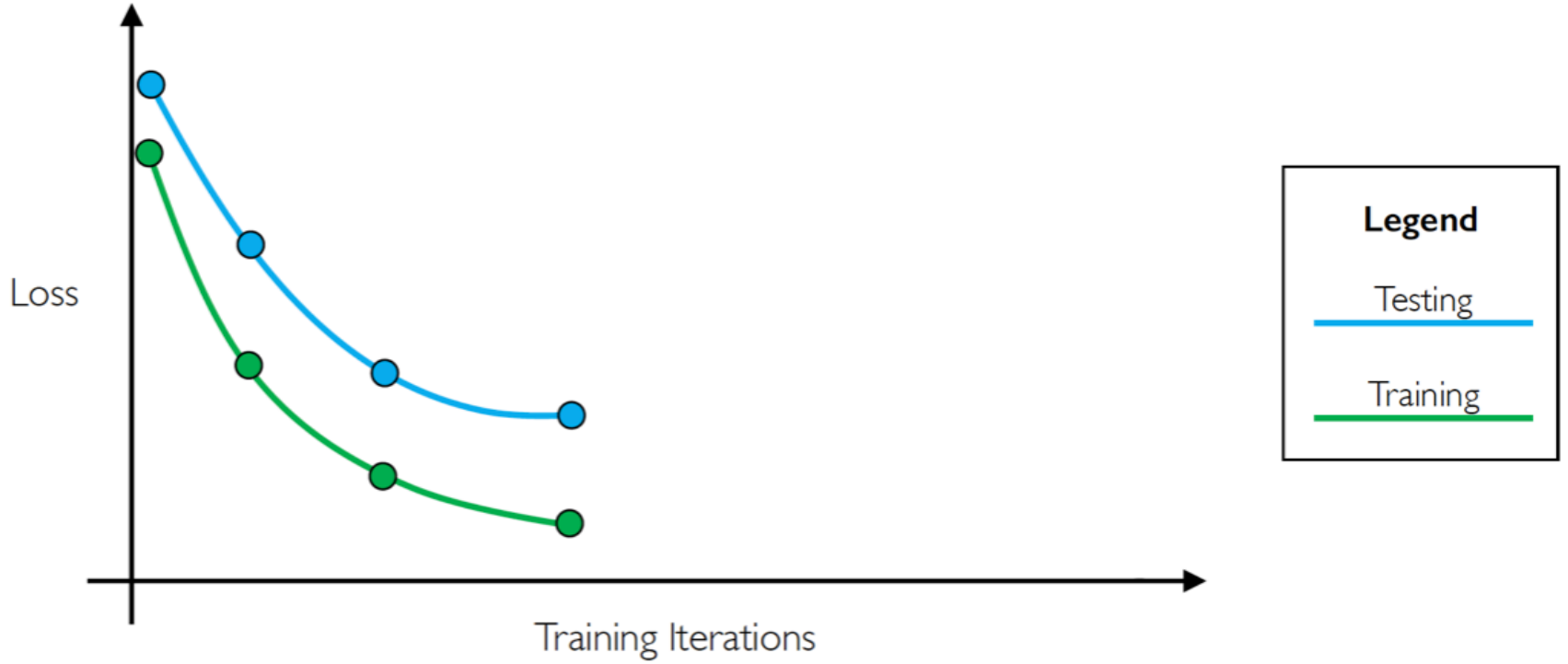
- Stop training before we have a chance to overfit

Regularization: early stopping

- Stop training before we have a chance to overfit

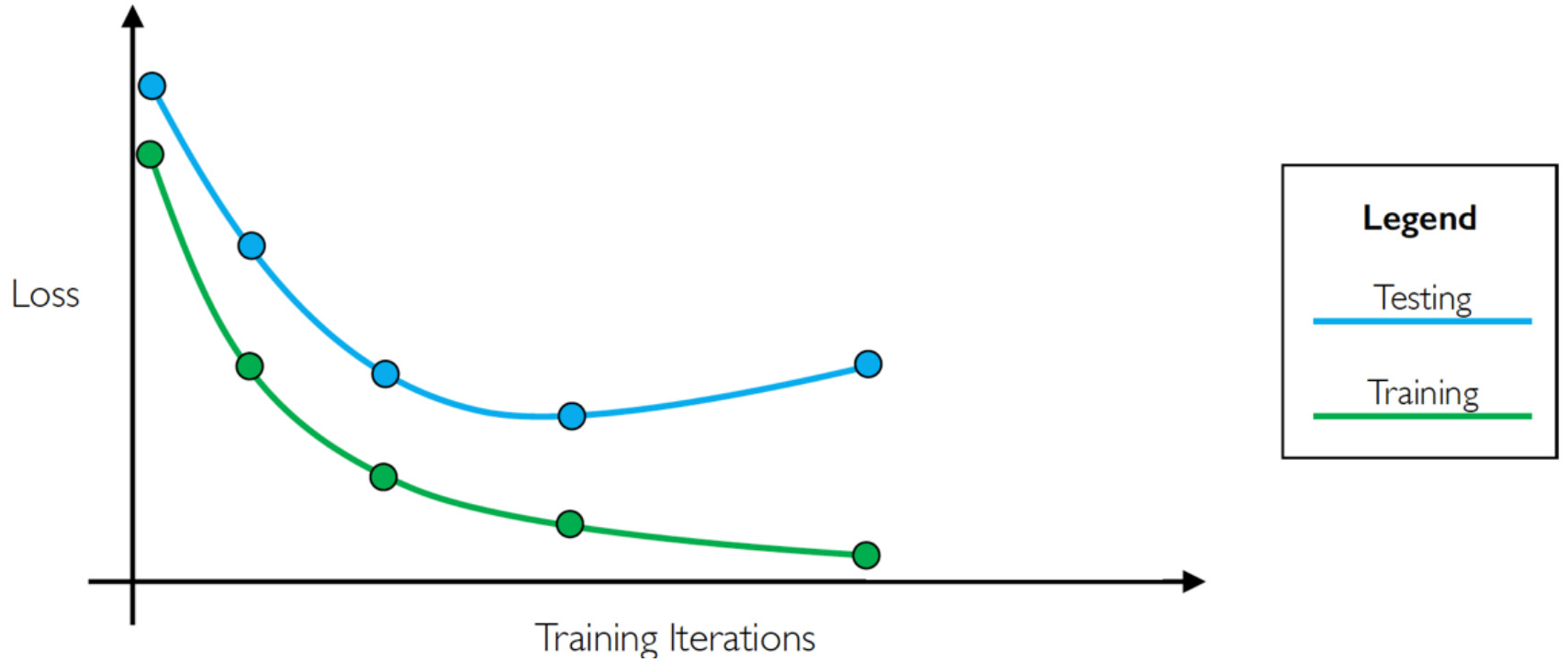
Regularization: early stopping

- Stop training before we have a chance to overfit



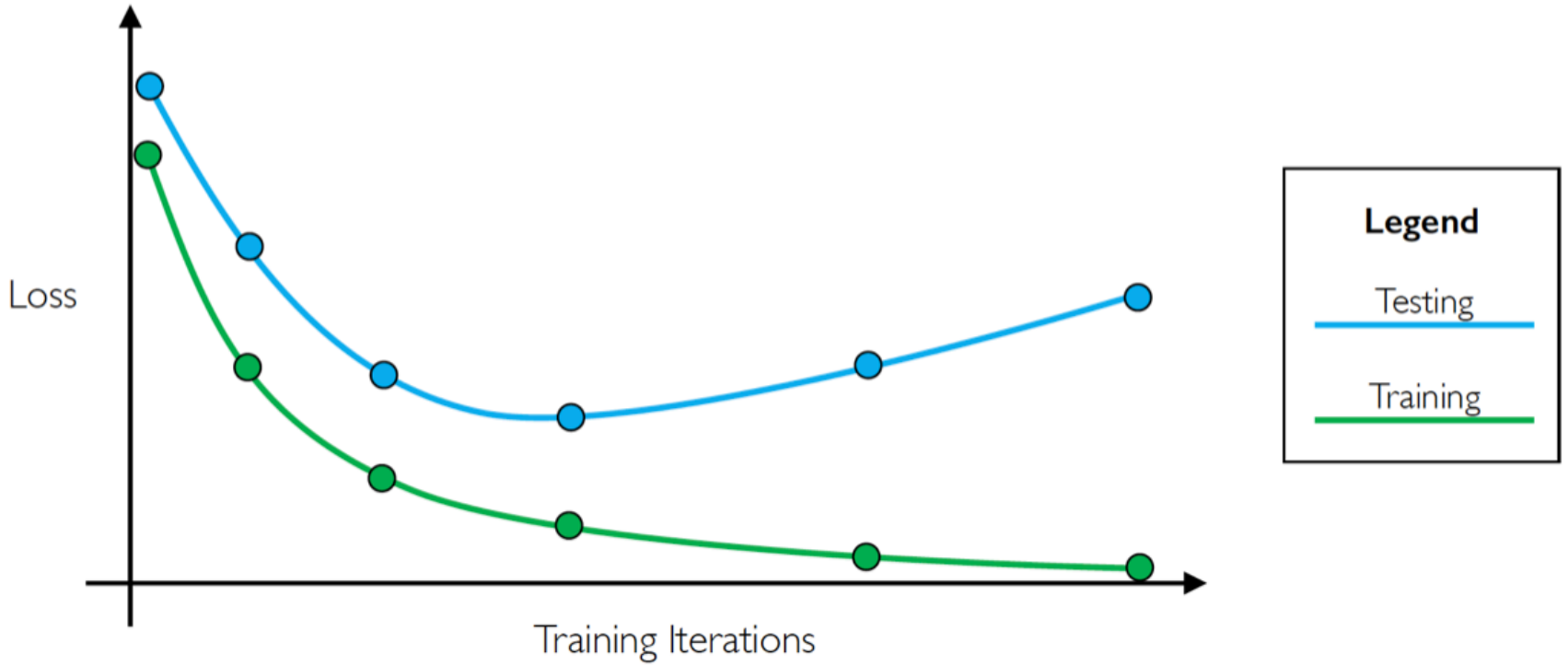
Regularization: early stopping

- Stop training before we have a chance to overfit



Regularization: early stopping

- Stop training before we have a chance to overfit



Regularization: early stopping

- Stop training before we have a chance to overfit

Regularization: early stopping

- Stop training before we have a chance to overfit

Regularization: early stopping

- Stop training before we have a chance to overfit

Regularization: early stopping

- PyTorch's early stopping is not Embedded with the library

- you could:
  - use program it by scratch: `05_Early_Stopping.ipynb`

  - or download an open-source:
    - https://github.com/ncullen93/torchsample
    - https://github.com/Bjarten/early-stopping-pytorch

Notebook

- Concise Implementation MLP